



MEDIATOR PATTERN

1 MỤC TIÊU BÀI HỌC

- Trình bày định nghĩa Mediator Pattern
- Phân tích được vấn đề coupling chéo giữa nhiều lớp
- Refactor hệ thống nhiều dependency chéo sang Mediator
- Thiết kế UML BEFORE và AFTER
- Cài đặt Mediator trong C#
- Áp dụng vào Order Management System

VẤN ĐỀ (PROBLEM) – Order Management System

Giả sử hệ thống có: OrderService, PaymentService, InventoryService, NotificationService, ShippingService

✗ Thiết kế ban đầu (Coupling chéo)

```
public class OrderService
{
    private PaymentService _payment;
    private InventoryService _inventory;
    private NotificationService _notification;
    private ShippingService _shipping;

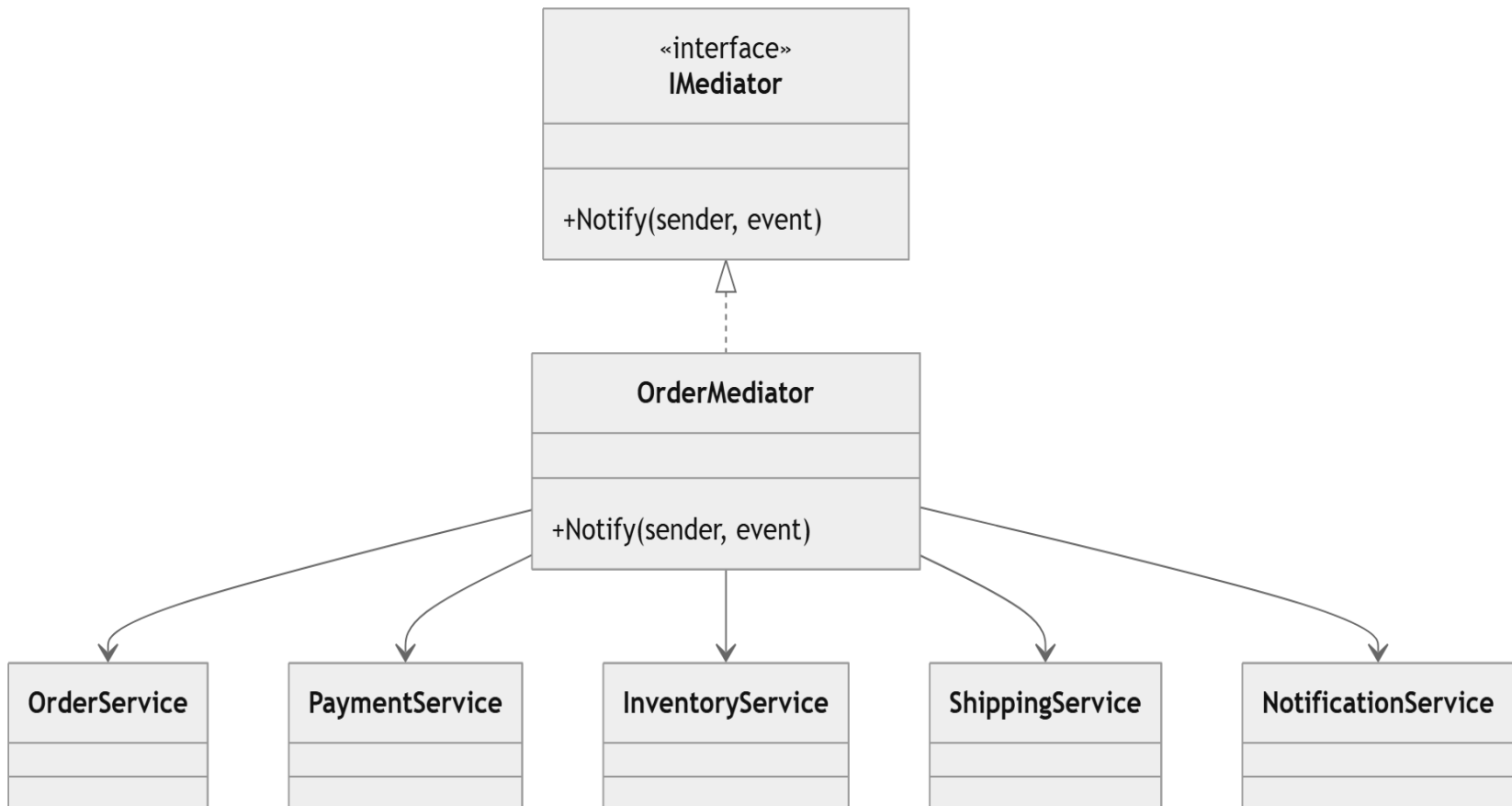
    public void PlaceOrder()
    {
        _payment.ProcessPayment();
        _inventory.ReserveStock();
        _shipping.CreateShipment();
        _notification.SendEmail();
    }
}
```

GIẢI PHÁP – MEDIATOR PATTERN

📌 Định nghĩa

“Định nghĩa một đối tượng đóng vai trò bao bọc (điều phối) cách mà một tập hợp các đối tượng tương tác với nhau.”

👉 Thay vì các service nói chuyện trực tiếp với nhau → tất cả giao tiếp qua Mediator.



TRIỂN KHAI MEDIATOR (Step 1 & 2)

Step 1: Mediator Interface

```
public interface IMediator
{
    void Notify(object sender, string eventCode);
}
```

Step 2: Concrete Mediator

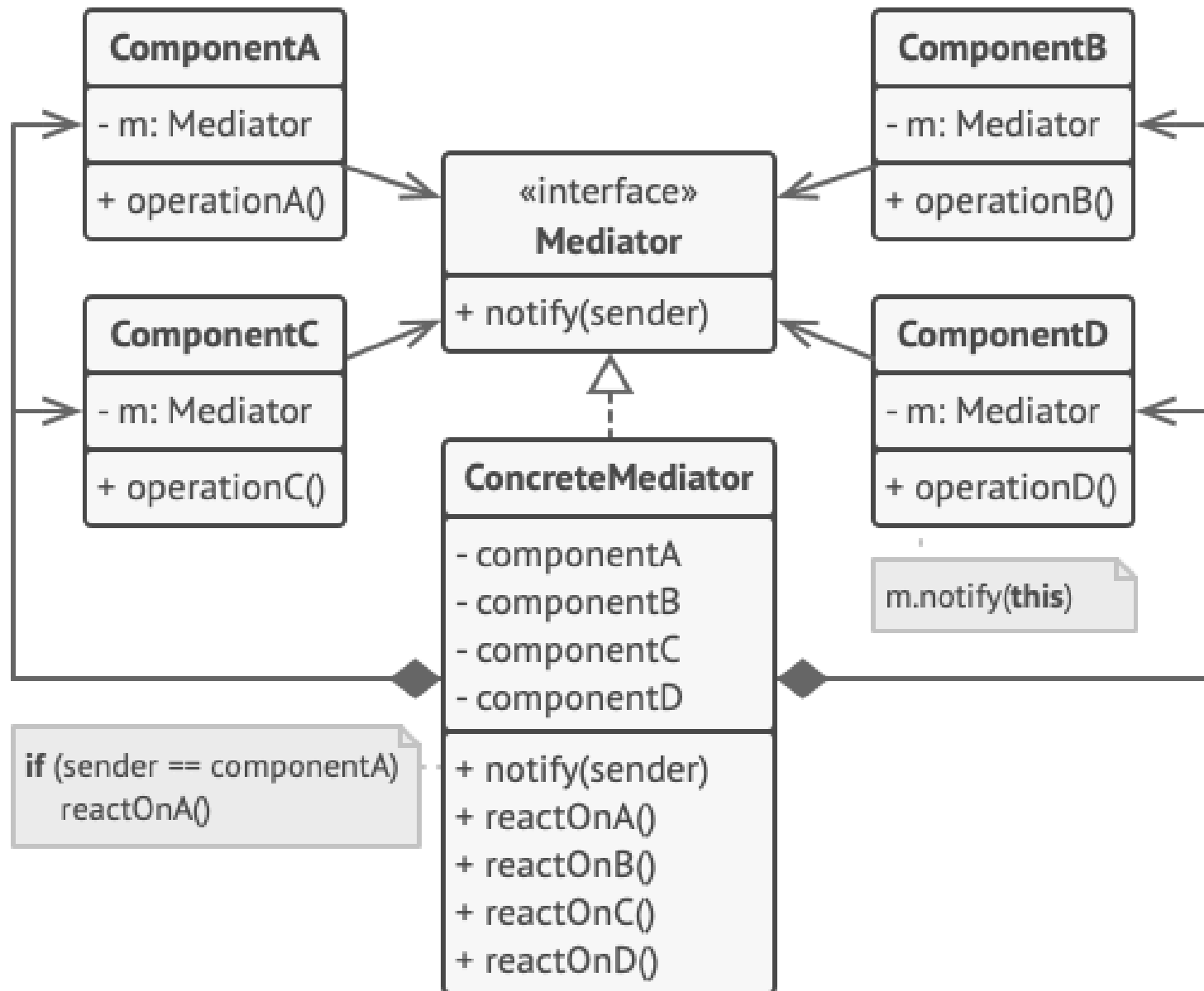
```
public class OrderMediator : IMediator
{
    private PaymentService _payment;
    private InventoryService _inventory;
    private ShippingService _shipping;
    private NotificationService _notification;
    // ... Constructor omitted for brevity ...
    public void Notify(object sender, string eventCode)
    {
        if (eventCode == "OrderPlaced")
        {
            _payment.ProcessPayment();
            _inventory.ReserveStock();
            _shipping.CreateShipment();
            _notification.SendEmail();
        }
    }
}
```

TRIỂN KHAI MEDIATOR (Step 3)

Step 3: OrderService chỉ thông báo

```
public class OrderService
{
    private readonly IMediator _mediator;
    public OrderService(IMediator mediator)
    {
        _mediator = mediator;
    }
    public void PlaceOrder()
    {
        Console.WriteLine("Order Created");
        _mediator.Notify(this, "OrderPlaced");
    }
}
```

Structure of MEDIATOR Pattern



SO SÁNH VÀ PHÂN BIỆT

BEFORE	AFTER
Coupling chéo	Centralized coordination
Khó mở rộng	Thêm service chỉ sửa Mediator
SRP vi phạm	Tách business & coordination
Circular dependency	Không còn

Pattern	Mục đích
Observer	Broadcast thông báo
Command	Đóng gói hành động
Strategy	Thay thuật toán
Mediator	Điều phối tương tác

BÀI TẬP THỰC HÀNH

 Bài 1 – Refactor: Vẽ UML BEFORE/AFTER, So sánh coupling.

 Bài 2 – Mở rộng hệ thống: Thêm LoyaltyService, LoggingService (Không sửa OrderService).

 Bài 3 – Nâng cao: Kết hợp Mediator + Command, Event Sourcing, Async mediator.

KẾT LUẬN

- ✓ Giảm coupling chéo
- ✓ Tách coordination logic
- ✓ Dễ mở rộng, tránh circular dependency
- ✓ Phù hợp Clean Architecture

SAU BÀI HỌC

Sinh viên hiểu cách điều phối hệ thống nhiều service.

Không thiết kế hệ thống kiểu “mọi lớp nói chuyện với mọi lớp”.